

EXPERIMENTS

9. Compare Linear and Polynomial Regression using Python

CODE:

```
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score
```

```
# -----
```

```
# Dataset 1: House Price Prediction
```

```
# -----
```

```
house_X = [[800], [1000], [1200], [1400], [1600],
            [1800], [2000], [2200], [2400], [2600]]
```

```
house_y = [32, 40, 52, 68, 88, 112, 140, 172, 208, 248]
```

```
house_new = [[1700]]
```

```
# -----
```

```
# Dataset 2: Student Study Hours vs Marks
```

```
# -----
```

```
student_X = [[1], [2], [3], [4], [5],
              [6], [7], [8], [9], [10]]
```

```
student_y = [18, 30, 43, 57, 68, 78, 86, 92, 96, 98]
```

```
student_new = [[7.5]]
```

```
# -----
```

```
# Dataset 3: Car Speed vs Fuel Consumption
```

```
# -----
```

```
car_X = [[40], [50], [60], [70], [80],
          [90], [100], [110], [120], [130]]
```

```

car_y = [4.5, 4.8, 5.2, 5.9, 6.8,
          8.0, 9.5, 11.2, 13.4, 16.0]
car_new = [[95]]

# -----
# Dataset 4: Advertising Cost vs Sales
# -----

ad_X = [[5], [10], [15], [20], [25],
         [30], [35], [40], [45], [50]]
ad_y = [1.2, 2.5, 4.1, 6.2, 8.8,
        11.9, 15.5, 19.6, 24.2, 29.3]
ad_new = [[32]]

# -----
# Dataset 5: Years of Experience vs Salary
# -----

salary_X = [[1], [2], [3], [4], [5],
            [6], [7], [8], [9], [10]]
salary_y = [3.0, 3.8, 4.9, 6.3, 8.0,
            10.2, 12.9, 16.1, 19.8, 24.0]
salary_new = [[7.5]]

# -----
# Function to perform Linear and Polynomial Regression
# -----

def compare_regression(name, X, y, new_sample):

    print("\n=====")

```

```
print(name)
print("=====")
```

```
# Linear Regression
```

```
linear_model = LinearRegression()
```

```
linear_model.fit(X, y)
```

```
linear_prediction = linear_model.predict(new_sample)
```

```
linear_train_prediction = linear_model.predict(X)
```

```
linear_mse = mean_squared_error(y, linear_train_prediction)
```

```
linear_r2 = r2_score(y, linear_train_prediction)
```

```
# Polynomial Features
```

```
poly = PolynomialFeatures(degree=2)
```

```
X_poly = poly.fit_transform(X)
```

```
new_sample_poly = poly.transform(new_sample)
```

```
# Polynomial Regression
```

```
poly_model = LinearRegression()
```

```
poly_model.fit(X_poly, y)
```

```
poly_prediction = poly_model.predict(new_sample_poly)
```

```
poly_train_prediction = poly_model.predict(X_poly)
```

```
poly_mse = mean_squared_error(y, poly_train_prediction)
```

```
poly_r2 = r2_score(y, poly_train_prediction)
```

```
# Display Results
```

```
print("New Sample:", new_sample[0][0])
```

```
print("\nLinear Regression")
```

```
print("Prediction :", round(linear_prediction[0], 2))
```

```
print("MSE      :", round(linear_mse, 4))
```

```
print("R2 Score  :", round(linear_r2, 4))
```

```
print("\nPolynomial Regression (Degree 2)")
```

```
print("Prediction :", round(poly_prediction[0], 2))
```

```
print("MSE      :", round(poly_mse, 4))
```

```
print("R2 Score  :", round(poly_r2, 4))
```

```
print("\nComparison")
```

```
if poly_r2 > linear_r2:
```

```
    print("Polynomial Regression gives better R2 Score.")
```

```
else:
```

```
    print("Linear Regression gives better R2 Score.")
```

```
# -----
```

```
# Run all 5 datasets
```

```
# -----
```

```
compare_regression(
```

```
    "Dataset 1: House Price Prediction",
```

```
    house_X, house_y, house_new
```

)

```
compare_regression(  
    "Dataset 2: Student Study Hours vs Marks",  
    student_X, student_y, student_new  
)
```

```
compare_regression(  
    "Dataset 3: Car Speed vs Fuel Consumption",  
    car_X, car_y, car_new  
)
```

```
compare_regression(  
    "Dataset 4: Advertising Cost vs Sales",  
    ad_X, ad_y, ad_new  
)
```

```
compare_regression(  
    "Dataset 5: Years of Experience vs Salary",  
    salary_X, salary_y, salary_new  
)
```

OUTPUT:

=====

Dataset 1: House Price Prediction

=====

New Sample: 1700

Linear Regression

Prediction : 116.0

MSE : 211.2

R2 Score : 0.9574

Polynomial Regression (Degree 2)

Prediction : 99.5

MSE : 0.0

R2 Score : 1.0

Comparison

Polynomial Regression gives better R2 Score.

Dataset 2: Student Study Hours vs Marks

New Sample: 7.5

Linear Regression

Prediction : 85.07

MSE : 31.6291

R2 Score : 0.957

Polynomial Regression (Degree 2)

Prediction : 88.29

MSE : 1.3261

R2 Score : 0.9982

Comparison

Polynomial Regression gives better R2 Score.

Dataset 3: Car Speed vs Fuel Consumption

New Sample: 95

Linear Regression

Prediction : 9.78

MSE : 1.0801

R2 Score : 0.9223

Polynomial Regression (Degree 2)

Prediction : 8.74

MSE : 0.0091

R2 Score : 0.9993

Comparison

Polynomial Regression gives better R2 Score.

Dataset 4: Advertising Cost vs Sales

New Sample: 32

Linear Regression

Prediction : 15.13

MSE : 3.1826

R2 Score : 0.9617

Polynomial Regression (Degree 2)

Prediction : 13.3

MSE : 0.0015

R2 Score : 1.0

Comparison

Polynomial Regression gives better R2 Score.

Dataset 5: Years of Experience vs Salary

New Sample: 7.5

Linear Regression

Prediction : 15.49

MSE : 2.5464

R2 Score : 0.9447

Polynomial Regression (Degree 2)

Prediction : 14.56

MSE : 0.0155

R2 Score : 0.9997

Comparison

Polynomial Regression gives better R2 Score.

10. Write a Python Program to Implement Expectation & Maximization Algorithm

CODE:

```
from sklearn.mixture import GaussianMixture

# =====
# Function to perform EM Clustering
# =====

def em_clustering(name, X, new_sample):

    print("\n=====")
    print(name)
    print("=====")

    # Create EM model using Gaussian Mixture Model
    gmm = GaussianMixture(
        n_components=2,
        random_state=42
    )

    # Train the model
    gmm.fit(X)

    # Predict cluster labels
    clusters = gmm.predict(X)

    # Predict cluster for new sample
```

```
new_cluster = gmm.predict(new_sample)
```

```
# Display results
```

```
print("Cluster Labels:")
```

```
print(clusters)
```

```
print("\nNew Sample:")
```

```
print(new_sample[0])
```

```
print("New Sample Cluster:")
```

```
print(new_cluster[0])
```

```
# =====
```

```
# Dataset 1: Student Performance Clustering
```

```
# =====
```

```
student_data = [
```

```
    [2, 65],
```

```
    [3, 70],
```

```
    [4, 75],
```

```
    [5, 80],
```

```
    [6, 85],
```

```
    [7, 88],
```

```
    [8, 90],
```

```
    [9, 93],
```

```
    [10, 95],
```

```
    [11, 98]
```

```
]
```

```
student_new = [[7, 89]]
```

```
em_clustering(  
    "Dataset 1: Student Performance Clustering",  
    student_data,  
    student_new  
)
```

```
# =====  
# Dataset 2: Customer Segmentation  
# =====
```

```
customer_data = [  
    [2, 20],  
    [3, 25],  
    [4, 30],  
    [5, 40],  
    [6, 50],  
    [8, 65],  
    [10, 75],  
    [12, 82],  
    [14, 90],  
    [16, 96]  
]
```

```
customer_new = [[9, 70]]
```

```
em_clustering(  
    "Dataset 2: Customer Segmentation",  
    customer_data,  
    customer_new  
)
```

```
# =====  
# Dataset 3: House Size Clustering  
# =====
```

```
house_data = [  
    [800, 30],  
    [900, 35],  
    [1000, 40],  
    [1200, 50],  
    [1400, 60],  
    [1600, 75],  
    [1800, 90],  
    [2000, 105],  
    [2200, 120],  
    [2400, 135]  
]
```

```
house_new = [[1700, 82]]
```

```
em_clustering(  
    "Dataset 3: House Size Clustering",  
    house_data,
```

```
house_new
)

# =====
# Dataset 4: Employee Salary Analysis
# =====

employee_data = [
    [1, 3.0],
    [2, 3.8],
    [3, 4.8],
    [4, 6.0],
    [5, 7.5],
    [6, 9.0],
    [7, 10.8],
    [8, 12.6],
    [9, 14.5],
    [10, 16.5]
]

employee_new = [[7, 11.0]]

em_clustering(
    "Dataset 4: Employee Salary Analysis",
    employee_data,
    employee_new
)
```

```
# =====  
# Dataset 5: Vehicle Speed Analysis  
# =====
```

```
vehicle_data = [  
    [40, 4.5],  
    [50, 5.0],  
    [60, 5.6],  
    [70, 6.5],  
    [80, 7.8],  
    [90, 9.2],  
    [100, 10.8],  
    [110, 12.5],  
    [120, 14.3],  
    [130, 16.2]  
]
```

```
vehicle_new = [[95, 10.0]]
```

```
em_clustering(  
    "Dataset 5: Vehicle Speed Analysis",  
    vehicle_data,  
    vehicle_new  
)
```

OUTPUT:

Dataset 1: Student Performance Clustering

Cluster Labels:

[0 0 0 0 0 1 1 1 1 1]

New Sample:

[7, 89]

New Sample Cluster:

1

Dataset 2: Customer Segmentation

Cluster Labels:

[0 0 0 0 0 0 1 1 1 1]

New Sample:

[9, 70]

New Sample Cluster:

0

Dataset 3: House Size Clustering

Cluster Labels:

[0 0 0 0 1 1 1 1 1 1]

New Sample:

[1700, 82]

New Sample Cluster:

1

Dataset 4: Employee Salary Analysis

Cluster Labels:

[0 0 0 0 0 1 1 1 1 1]

New Sample:

[7, 11.0]

New Sample Cluster:

1

Dataset 5: Vehicle Speed Analysis

Cluster Labels:

[0 0 0 0 0 0 1 1 1 1]

New Sample:

[95, 10.0]

New Sample Cluster:

0

11. To classify credit scores using the Decision Tree algorithm.

CODE:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder

# =====
# Function for Decision Tree Classification
# =====

def credit_score_classification(name, X, y, new_sample):

    print("\n=====")
    print(name)
    print("=====")

    # Create and train Decision Tree
    model = DecisionTreeClassifier(
        criterion="entropy",
        random_state=42
    )

    model.fit(X, y)

    # Predict credit score
    prediction = model.predict(new_sample)

    print("New Sample:", new_sample[0])
    print("Predicted Credit Score:", prediction[0])
```

```
# =====
```

```
# Dataset 1: Customer Loan Profile
```

```
# =====
```

```
# EMI: Yes = 1, No = 0
```

```
X1 = [  
    [12, 10, 1, 1],  
    [10, 8, 2, 1],  
    [8, 6, 2, 1],  
    [6, 4, 3, 0],  
    [4, 2, 4, 0],  
    [15, 12, 1, 1],  
    [7, 5, 3, 1],  
    [5, 3, 4, 0],  
    [11, 9, 2, 1],  
    [3, 1, 5, 0]  
]
```

```
y1 = [  
    "Excellent",  
    "Good",  
    "Good",  
    "Average",  
    "Poor",  
    "Excellent",  
    "Average",  
    "Poor",  
]
```

```
"Good",  
"Poor"  
]
```

```
# New sample: 9, 7, 2, Yes  
new1 = [[9, 7, 2, 1]]
```

```
credit_score_classification(  
    "Dataset 1: Customer Loan Profile",  
    X1, y1, new1  
)
```

```
# =====  
# Dataset 2: Bank Customer Credit Rating  
# =====
```

```
# Loan Default: No = 0, Yes = 1
```

```
X2 = [  
    [90000, 45, 5, 0],  
    [75000, 40, 6, 0],  
    [60000, 35, 8, 0],  
    [45000, 30, 10, 1],  
    [30000, 25, 12, 1],  
    [95000, 48, 4, 0],  
    [65000, 37, 7, 0],  
    [40000, 28, 9, 1],  
    [28000, 24, 13, 1],  
    [85000, 43, 5, 0]
```

```
]
```

```
y2 = [  
    "Excellent",  
    "Good",  
    "Good",  
    "Average",  
    "Poor",  
    "Excellent",  
    "Good",  
    "Average",  
    "Poor",  
    "Excellent"  
]
```

```
# New sample: 70000, 38, 6, No
```

```
new2 = [[70000, 38, 6, 0]]
```

```
credit_score_classification(  
    "Dataset 2: Bank Customer Credit Rating",  
    X2, y2, new2  
)
```

```
# =====
```

```
# Dataset 3: Credit Card Customer Analysis
```

```
# =====
```

```
X3 = [
```

```
[8, 40000, 0],  
[7, 35000, 2],  
[6, 30000, 4],  
[5, 25000, 10],  
[4, 20000, 20],  
[9, 45000, 0],  
[6, 28000, 5],  
[5, 23000, 12],  
[3, 18000, 25],  
[8, 42000, 1]  
]
```

```
y3 = [  
    "Excellent",  
    "Good",  
    "Good",  
    "Average",  
    "Poor",  
    "Excellent",  
    "Good",  
    "Average",  
    "Poor",  
    "Excellent"  
]
```

```
# New sample: 7, 36000, 3
```

```
new3 = [[7, 36000, 3]]
```

```
credit_score_classification(  

```

```
"Dataset 3: Credit Card Customer Analysis",  
X3, y3, new3  
)
```

```
# =====
```

```
# Dataset 4: Personal Finance Profile
```

```
# =====
```

```
X4 = [  
    [10, 30000, 1],  
    [8, 35000, 2],  
    [6, 40000, 3],  
    [4, 45000, 5],  
    [2, 50000, 7],  
    [11, 28000, 1],  
    [7, 36000, 2],  
    [5, 43000, 4],  
    [3, 48000, 6],  
    [9, 32000, 1]  
]
```

```
y4 = [  
    "Excellent",  
    "Good",  
    "Good",  
    "Average",  
    "Poor",  
    "Excellent",
```

```
"Good",  
"Average",  
"Poor",  
"Excellent"  
]
```

```
# New sample: 7, 34000, 2
```

```
new4 = [[7, 34000, 2]]
```

```
credit_score_classification(  
    "Dataset 4: Personal Finance Profile",  
    X4, y4, new4  
)
```

```
# =====  
# Dataset 5: Home Loan Eligibility  
# =====
```

```
# Loan Status:  
# Approved = 0  
# Pending = 1  
# Rejected = 2
```

```
X5 = [  
    [15, 850, 1, 0],  
    [12, 800, 2, 0],  
    [10, 760, 2, 0],  
    [8, 700, 3, 1],
```

```
[6, 620, 4, 2],  
[16, 880, 1, 0],  
[11, 780, 2, 0],  
[7, 680, 3, 1],  
[5, 600, 5, 2],  
[14, 830, 1, 0]  
]
```

```
y5 = [  
    "Excellent",  
    "Good",  
    "Good",  
    "Average",  
    "Poor",  
    "Excellent",  
    "Good",  
    "Average",  
    "Poor",  
    "Excellent"  
]
```

```
# New sample: 11, 790, 2, Approved
```

```
new5 = [[11, 790, 2, 0]]
```

```
credit_score_classification(  
    "Dataset 5: Home Loan Eligibility",  
    X5, y5, new5  
)
```

OUTPUT:

Dataset 1: Customer Loan Profile

New Sample: [9, 7, 2, 1]

Predicted Credit Score: Good

Dataset 2: Bank Customer Credit Rating

New Sample: [70000, 38, 6, 0]

Predicted Credit Score: Good

Dataset 3: Credit Card Customer Analysis

New Sample: [7, 36000, 3]

Predicted Credit Score: Good

Dataset 4: Personal Finance Profile

New Sample: [7, 34000, 2]

Predicted Credit Score: Good

Dataset 5: Home Loan Eligibility

New Sample: [11, 790, 2, 0]

Predicted Credit Score: Good

12. To classify Iris flowers using the K-Nearest Neighbours (KNN) algorithm.

CODE:

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split

# =====
# Function for KNN Classification
# =====

def knn_classification(name, X, y, new_sample):

    print("\n=====")
    print(name)
    print("=====")

    # Split the dataset
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=42
    )

    # Create KNN model
    knn = KNeighborsClassifier(n_neighbors=3)

    # Train the model
    knn.fit(X_train, y_train)

    # Predict new sample
```

```
prediction = knn.predict(new_sample)
```

```
# Display result
```

```
print("New Sample:", new_sample[0])
```

```
print("Predicted Class:", prediction[0])
```

```
# =====
```

```
# Dataset 1: Fruit Classification
```

```
# =====
```

```
X1 = [
```

```
    [150, 7.5, 90],
```

```
    [160, 7.8, 88],
```

```
    [145, 7.3, 91],
```

```
    [120, 5.5, 70],
```

```
    [125, 5.8, 72],
```

```
    [130, 6.0, 74],
```

```
    [155, 7.6, 89],
```

```
    [118, 5.4, 69],
```

```
    [148, 7.4, 92],
```

```
    [122, 5.7, 71]
```

```
]
```

```
y1 = [
```

```
    "Apple",
```

```
    "Apple",
```

```
    "Apple",
```

```
    "Orange",
```

```
"Orange",  
"Orange",  
"Apple",  
"Orange",  
"Apple",  
"Orange"  
]
```

```
new1 = [[152, 7.5, 90]]
```

```
knn_classification(  
    "Dataset 1: Fruit Classification",  
    X1, y1, new1  
)
```

```
# =====  
# Dataset 2: Student Placement Classification  
# =====
```

```
X2 = [  
    [9.2, 90, 88],  
    [8.8, 85, 82],  
    [8.5, 80, 79],  
    [7.2, 72, 70],  
    [6.8, 68, 65],  
    [9.0, 92, 90],  
    [7.5, 75, 74],  
    [8.6, 84, 81],
```

```
[6.5, 60, 58],  
[8.9, 88, 86]  
]
```

```
y2 = [  
    "Placed",  
    "Placed",  
    "Placed",  
    "Not Placed",  
    "Not Placed",  
    "Placed",  
    "Not Placed",  
    "Placed",  
    "Not Placed",  
    "Placed"  
]
```

```
new2 = [[8.7, 86, 84]]
```

```
knn_classification(  
    "Dataset 2: Student Placement Classification",  
    X2, y2, new2  
)
```

```
# =====
```

```
# Dataset 3: Disease Diagnosis
```

```
# =====
```

]

]

```
new3 = [[38.7, 109, 94]]
```

knn_classification(

```
"Dataset 3: Disease Diagnosis",  
X3, y3, new3  
)
```

```
# =====
```

```
# Dataset 4: Employee Promotion Prediction
```

```
# =====
```

```
X4 = [  
    [10, 95, 50],  
    [8, 90, 45],  
    [7, 88, 40],  
    [3, 65, 20],  
    [2, 60, 18],  
    [9, 93, 48],  
    [4, 70, 25],  
    [8, 89, 42],  
    [2, 58, 15],  
    [6, 85, 38]  
]
```

```
y4 = [  
    "Promoted",  
    "Promoted",  
    "Promoted",  
    "Not Promoted",  
    "Not Promoted",  
    "Promoted",
```

```
"Not Promoted",  
"Promoted",  
"Not Promoted",  
"Promoted"  
]
```

```
new4 = [[7, 90, 44]]
```

```
knn_classification(  
    "Dataset 4: Employee Promotion Prediction",  
    X4, y4, new4  
)
```

```
# =====  
# Dataset 5: Vehicle Classification  
# =====
```

```
X5 = [  
    [100, 95, 70],  
    [125, 110, 65],  
    [150, 125, 55],  
    [800, 850, 22],  
    [1000, 950, 20],  
    [1200, 1050, 18],  
    [110, 100, 68],  
    [900, 900, 21],  
    [140, 120, 58],  
    [1300, 1100, 17]
```

```
]
```

```
y5 = [  
    "Bike",  
    "Bike",  
    "Bike",  
    "Car",  
    "Car",  
    "Car",  
    "Bike",  
    "Car",  
    "Bike",  
    "Car"  
]
```

```
new5 = [[135, 118, 60]]
```

```
knn_classification(  
    "Dataset 5: Vehicle Classification",  
    X5, y5, new5  
)
```

OUTPUT:

```
=====
```

Dataset 1: Fruit Classification

```
=====
```

New Sample: [152, 7.5, 90]

Predicted Class: Apple

Dataset 2: Student Placement Classification

New Sample: [8.7, 86, 84]

Predicted Class: Placed

Dataset 3: Disease Diagnosis

New Sample: [38.7, 109, 94]

Predicted Class: Positive

Dataset 4: Employee Promotion Prediction

New Sample: [7, 90, 44]

Predicted Class: Promoted

Dataset 5: Vehicle Classification

New Sample: [135, 118, 60]

Predicted Class: Bike

13. To predict car prices using the Linear Regression algorithm.

CODE:

```
from sklearn.linear_model import LinearRegression

# =====
# Function for Linear Regression
# =====

def car_price_prediction(name, X, y, new_sample):

    print("\n=====")
    print(name)
    print("=====")

    # Create Linear Regression model
    model = LinearRegression()

    # Train the model
    model.fit(X, y)

    # Predict car price
    prediction = model.predict(new_sample)

    # Display result
    print("New Sample:", new_sample[0])
    print("Predicted Car Price: ₹", round(prediction[0], 2), "Lakhs")
```

```
# =====
```

```
# Dataset 1: Basic Car Price Prediction
```

```
# =====
```

```
X1 = [  
    [1, 12000, 1200],  
    [2, 18000, 1200],  
    [3, 25000, 1300],  
    [4, 35000, 1400],  
    [5, 45000, 1500],  
    [6, 55000, 1500],  
    [7, 65000, 1600],  
    [8, 75000, 1600],  
    [9, 85000, 1800],  
    [10, 95000, 1800]  
]
```

```
y1 = [  
    8.5, 8.0, 7.4, 6.8, 6.1,  
    5.5, 5.0, 4.5, 4.0, 3.5  
]
```

```
new1 = [[4, 38000, 1400]]
```

```
car_price_prediction(  
    "Dataset 1: Basic Car Price Prediction",  
    X1, y1, new1  
)
```

```
# =====
```

```
# Dataset 2: Used Car Price Prediction
```

```
# =====
```

```
X2 = [  
    [2024, 10000, 22],  
    [2023, 18000, 21],  
    [2022, 25000, 20],  
    [2021, 35000, 19],  
    [2020, 45000, 18],  
    [2019, 55000, 17],  
    [2018, 65000, 16],  
    [2017, 75000, 15],  
    [2016, 85000, 14],  
    [2015, 95000, 13]  
]
```

```
y2 = [  
    10.5, 9.8, 9.0, 8.2, 7.5,  
    6.8, 6.0, 5.4, 4.8, 4.2  
]
```

```
new2 = [[2021, 32000, 19]]
```

```
car_price_prediction(  
    "Dataset 2: Used Car Price Prediction",  
    X2, y2, new2  
)
```

```
# =====  
# Dataset 3: Sedan Car Price Prediction  
# =====
```

```
X3 = [  
    [1000, 70, 10000],  
    [1200, 82, 15000],  
    [1400, 95, 22000],  
    [1500, 105, 30000],  
    [1600, 115, 38000],  
    [1800, 130, 45000],  
    [2000, 145, 52000],  
    [2200, 160, 60000],  
    [2400, 175, 68000],  
    [2600, 190, 76000]  
]
```

```
y3 = [  
    6.5, 7.4, 8.8, 9.8, 10.5,  
    11.8, 13.2, 14.5, 15.8, 17.2  
]
```

```
new3 = [[1700, 120, 42000]]
```

```
car_price_prediction(  
    "Dataset 3: Sedan Car Price Prediction",  
    X3, y3, new3
```

)

```
# =====  
# Dataset 4: SUV Price Prediction  
# =====
```

```
X4 = [  
    [1500, 1, 12000],  
    [1800, 2, 20000],  
    [2000, 3, 28000],  
    [2200, 4, 36000],  
    [2400, 5, 44000],  
    [2600, 6, 52000],  
    [2800, 7, 60000],  
    [3000, 8, 68000],  
    [3200, 9, 76000],  
    [3400, 10, 84000]  
]
```

```
y4 = [  
    15.5, 14.8, 13.9, 13.0, 12.0,  
    11.0, 10.2, 9.4, 8.7, 8.0  
]
```

```
new4 = [[2100, 4, 34000]]
```

```
car_price_prediction(  
    "Dataset 4: SUV Price Prediction",
```

```
X4, y4, new4
)
```

```
# =====
# Dataset 5: Electric Car Price Prediction
# =====
```

```
X5 = [
    [30, 250, 8],
    [35, 300, 8],
    [40, 350, 7],
    [45, 400, 7],
    [50, 450, 6],
    [55, 500, 6],
    [60, 550, 5],
    [65, 600, 5],
    [70, 650, 4],
    [75, 700, 4]
]
```

```
y5 = [
    10.5, 12.0, 14.0, 16.5, 18.5,
    21.0, 24.0, 27.5, 31.0, 35.0
]
```

```
new5 = [[52, 470, 6]]
```

```
car_price_prediction(
```

```
"Dataset 5: Electric Car Price Prediction",  
X5, y5, new5  
)
```

OUTPUT:

Dataset 1: Basic Car Price Prediction

New Sample: [4, 38000, 1400]

Predicted Car Price: ₹ 6.8 Lakhs

Dataset 2: Used Car Price Prediction

New Sample: [2021, 32000, 19]

Predicted Car Price: ₹ 8.06 Lakhs

Dataset 3: Sedan Car Price Prediction

New Sample: [1700, 120, 42000]

Predicted Car Price: ₹ 11.14 Lakhs

Dataset 4: SUV Price Prediction

New Sample: [2100, 4, 34000]

Predicted Car Price: ₹ 13.15 Lakhs

Dataset 5: Electric Car Price Prediction

New Sample: [52, 470, 6]

Predicted Car Price: ₹ 20.72 Lakhs

14. To predict house prices using the Linear Regression algorithm.

CODE:

```
from sklearn.linear_model import LinearRegression
```

```
# =====  
# Function for Linear Regression  
# =====
```

```
def house_price_prediction(name, X, y, new_sample):
```

```
    print("\n=====")  
    print(name)  
    print("=====")
```

```
    # Create Linear Regression model
```

```
    model = LinearRegression()
```

```
    # Train the model
```

```
    model.fit(X, y)
```

```
    # Predict house price
```

```
    prediction = model.predict(new_sample)
```

```
# Display result

print("New Sample:", new_sample[0])

print("Predicted House Price: ₹",
      round(prediction[0], 2), "Lakhs")
```

```
# =====

# Dataset 1: House Size, Bedrooms, and Age

# =====
```

```
X1 = [
    [800, 2, 15],
    [1000, 2, 12],
    [1200, 3, 10],
    [1400, 3, 8],
    [1600, 4, 6],
    [1800, 4, 5],
    [2000, 5, 4],
    [2200, 5, 3],
    [2400, 6, 2],
    [2600, 6, 1]
]
```

```
y1 = [
    35, 42, 52, 63, 75,
    88, 102, 118, 135, 150
]
```

```
new1 = [[1700, 4, 5]]
```

```
house_price_prediction(  
    "Dataset 1: House Size, Bedrooms, and Age",  
    X1, y1, new1  
)
```

```
# =====  
# Dataset 2: House Area, Distance from City, and Parking  
# =====
```

```
X2 = [  
    [900, 20, 1],  
    [1100, 18, 1],  
    [1300, 16, 2],  
    [1500, 14, 2],  
    [1700, 12, 2],  
    [1900, 10, 3],  
    [2100, 8, 3],  
    [2300, 6, 4],  
    [2500, 5, 4],  
    [2700, 3, 5]  
]
```

```
y2 = [  
    40, 48, 60, 74, 88,  
    104, 122, 142, 165, 190  
]
```

```
new2 = [[1800, 11, 3]]
```

```
house_price_prediction(  
    "Dataset 2: House Area, Distance from City, and Parking",  
    X2, y2, new2  
)
```

```
# =====  
# Dataset 3: House Area, Floors, and Garden  
# =====
```

```
X3 = [  
    [1000, 1, 100],  
    [1200, 1, 120],  
    [1400, 2, 150],  
    [1600, 2, 180],  
    [1800, 2, 200],  
    [2000, 3, 240],  
    [2200, 3, 280],  
    [2400, 3, 320],  
    [2600, 4, 350],  
    [2800, 4, 400]  
]
```

```
y3 = [  
    45, 55, 68, 82, 96,  
    112, 130, 150, 172, 196
```

```
]
```

```
new3 = [[2100, 3, 250]]
```

```
house_price_prediction(  
    "Dataset 3: House Area, Floors, and Garden",  
    X3, y3, new3  
)
```

```
# =====  
# Dataset 4: Apartment Price Prediction  
# =====
```

```
X4 = [  
    [700, 1, 15],  
    [850, 2, 12],  
    [1000, 3, 10],  
    [1150, 4, 8],  
    [1300, 5, 7],  
    [1450, 6, 5],  
    [1600, 7, 4],  
    [1750, 8, 3],  
    [1900, 9, 2],  
    [2050, 10, 1]  
]
```

```
y4 = [  
    30, 38, 48, 60, 72,
```

```
86, 100, 116, 134, 154  
]
```

```
new4 = [[1500, 6, 4]]
```

```
house_price_prediction(  
    "Dataset 4: Apartment Price Prediction",  
    X4, y4, new4  
)
```

```
# =====  
# Dataset 5: Independent House Price Prediction  
# =====
```

```
X5 = [  
    [1200, 900, 2],  
    [1400, 1100, 2],  
    [1600, 1300, 3],  
    [1800, 1500, 3],  
    [2000, 1700, 4],  
    [2200, 1900, 4],  
    [2400, 2100, 5],  
    [2600, 2300, 5],  
    [2800, 2500, 6],  
    [3000, 2700, 6]  
]
```

```
y5 = [  
    1.2,  
    1.4,  
    1.6,  
    1.8,  
    2.0,  
    2.2,  
    2.4,  
    2.6,  
    2.8,  
    3.0  
]
```

```
48, 60, 74, 90, 108,  
128, 150, 174, 200, 228  
]
```

```
new5 = [[2300, 2000, 4]]
```

```
house_price_prediction(  
    "Dataset 5: Independent House Price Prediction",  
    X5, y5, new5  
)
```

OUTPUT:

Dataset 1: House Size, Bedrooms, and Age

New Sample: [1700, 4, 5]

Predicted House Price: ₹ 78.85 Lakhs

Dataset 2: House Area, Distance from City, and Parking

New Sample: [1800, 11, 3]

Predicted House Price: ₹ 102.33 Lakhs

Dataset 3: House Area, Floors, and Garden

New Sample: [2100, 3, 250]

Predicted House Price: ₹ 117.79 Lakhs

Dataset 4: Apartment Price Prediction

New Sample: [1500, 6, 4]

Predicted House Price: ₹ 87.03 Lakhs

Dataset 5: Independent House Price Prediction

New Sample: [2300, 2000, 4]

Predicted House Price: ₹ 146.0 Lakhs

15. To classify Iris flowers using the Naïve Bayes algorithm.

CODE:

```
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
```

```
# =====
# 1. FRUIT CLASSIFICATION
# =====
```

```
X = [
    [150, 7.5, 90],
    [160, 7.8, 88],
    [145, 7.3, 91],
    [120, 5.5, 70],
    [125, 5.8, 72],
    [130, 6.0, 74],
```

```
[155, 7.6, 89],  
[118, 5.4, 69],  
[148, 7.4, 92],  
[122, 5.7, 71]  
]
```

```
y = [  
    "Apple", "Apple", "Apple", "Orange", "Orange",  
    "Orange", "Apple", "Orange", "Apple", "Orange"  
]
```

```
new_sample = [[152, 7.5, 90]]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42  
)
```

```
model = GaussianNB()  
model.fit(X_train, y_train)
```

```
prediction = model.predict(new_sample)
```

```
print("===== FRUIT CLASSIFICATION =====")  
print("Predicted Class:", prediction[0])
```

```
# =====  
  
# 2. STUDENT PLACEMENT CLASSIFICATION  
  
# =====
```

```
X = [  
    [9.2, 90, 88],  
    [8.8, 85, 82],  
    [8.5, 80, 79],  
    [7.2, 72, 70],  
    [6.8, 68, 65],  
    [9.0, 92, 90],  
    [7.5, 75, 74],  
    [8.6, 84, 81],  
    [6.5, 60, 58],  
    [8.9, 88, 86]  
]
```

```
y = [  
    "Placed", "Placed", "Placed", "Not Placed", "Not Placed",  
    "Placed", "Not Placed", "Placed", "Not Placed", "Placed"  
]
```

```
new_sample = [[8.7, 86, 84]]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42  
)
```

```
model = GaussianNB()  
model.fit(X_train, y_train)
```

```
prediction = model.predict(new_sample)
```

```
print("\n===== STUDENT PLACEMENT =====")
```

```
print("Predicted Class:", prediction[0])
```

```
# =====
```

```
# 3. DISEASE DIAGNOSIS
```

```
# =====
```

```
X = [
```

```
    [39.0, 110, 94],
```

```
    [38.5, 108, 95],
```

```
    [37.0, 78, 99],
```

```
    [39.2, 115, 93],
```

```
    [36.8, 75, 98],
```

```
    [38.8, 112, 94],
```

```
    [37.2, 80, 98],
```

```
    [39.1, 114, 92],
```

```
    [36.7, 74, 99],
```

```
    [38.9, 111, 93]
```

```
]
```

```
y = [
```

```
    "Positive", "Positive", "Negative", "Positive", "Negative",
```

```
    "Positive", "Negative", "Positive", "Negative", "Positive"
```

```
]
```

```
new_sample = [[38.7, 109, 94]]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
model = GaussianNB()
model.fit(X_train, y_train)
```

```
prediction = model.predict(new_sample)
```

```
print("\n===== DISEASE DIAGNOSIS =====")
print("Predicted Class:", prediction[0])
```

```
# =====
# 4. EMPLOYEE PROMOTION PREDICTION
# =====
```

```
X = [
    [10, 95, 50],
    [8, 90, 45],
    [7, 88, 40],
    [3, 65, 20],
    [2, 60, 18],
    [9, 93, 48],
    [4, 70, 25],
    [8, 89, 42],
    [2, 58, 15],
    [6, 85, 38]
]
```

```
y = [  
    "Promoted", "Promoted", "Promoted", "Not Promoted",  
    "Not Promoted", "Promoted", "Not Promoted", "Promoted",  
    "Not Promoted", "Promoted"  
]
```

```
new_sample = [[7, 90, 44]]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42  
)
```

```
model = GaussianNB()  
model.fit(X_train, y_train)
```

```
prediction = model.predict(new_sample)
```

```
print("\n===== EMPLOYEE PROMOTION =====")  
print("Predicted Class:", prediction[0])
```

```
# =====  
# 5. VEHICLE CLASSIFICATION  
# =====
```

```
X = [  
    [100, 95, 70],  
    [125, 110, 65],
```

```
[150, 125, 55],  
[800, 850, 22],  
[1000, 950, 20],  
[1200, 1050, 18],  
[110, 100, 68],  
[900, 900, 21],  
[140, 120, 58],  
[1300, 1100, 17]  
]
```

```
y = [  
    "Bike", "Bike", "Bike", "Car", "Car",  
    "Car", "Bike", "Car", "Bike", "Car"  
]
```

```
new_sample = [[135, 118, 60]]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42  
)
```

```
model = GaussianNB()  
model.fit(X_train, y_train)
```

```
prediction = model.predict(new_sample)
```

```
print("\n===== VEHICLE CLASSIFICATION =====")  
print("Predicted Class:", prediction[0])
```

OUTPUT:

===== FRUIT CLASSIFICATION =====

Predicted Class: Apple

===== STUDENT PLACEMENT =====

Predicted Class: Placed

===== DISEASE DIAGNOSIS =====

Predicted Class: Positive

===== EMPLOYEE PROMOTION =====

Predicted Class: Promoted

===== VEHICLE CLASSIFICATION =====

Predicted Class: Bike